LAPORAN PRAKTIKUM HDFS



Dosen Pengampu:

Ersha Aisyah Elfaiz, S.Si., M.T.I.

Disusun oleh:

Muhammad Anwar Ismail (24051204161)

Program Studi Teknik Informatika

Fakultas Teknik

Universitas Negeri Surabaya

2026

DAFTAR ISI

DAFTAR ISI ..... **i**
DAFTAR GAMBAR **1**
BAB I PENDAHULUAN **3**
1.1 Tujuan Praktikum 3
BAB II DASAR TEORI **4**
2.1 Konsep Apache Hadoop & HDFS 4
2.2 NameNode, DataNode & Replication Factor 4
BAB III PEMBAHASAN **5**
3.1 Langkah-Langkah Praktikum dan Command 5
3.2 Hasil Eksekusi dan Screenshot 9
3.3 Jawaban Pertanyaan Analisis 23
BAB IV KESIMPULAN **26**
4.1 Kesimpulan 26
LAMPIRAN A COMMAND UTAMA **27**
LAMPIRAN B PORT YANG DIGUNAKAN **29**

DAFTAR GAMBAR

Gambar 3.2.1 - Menunjukkan struktur folder dan file yang berhasil dibuat.png 9
Gambar 3.2.2 - Validasi Docker Compose_1.png 10
Gambar 3.2.3 - Validasi Docker Compose_2.png 10
Gambar 3.2.4 - Proses Download Image_1.png.....11
Gambar 3.2.5 - Proses Download Image_2.png.....11
Gambar 3.2.6 - Menjalankan Cluster dan Verifikasi.png 12
Gambar 3.2.7 - NameNode Web UI_1.png..... 13
Gambar 3.2.8 - NameNode Web UI_2.png..... 13
Gambar 3.2.9 - DataNode Web UI.png 14
Gambar 3.2.10 - Melihat isi root HDFS.png..... 15
Gambar 3.2.11 - Membuat folder praktikum dan data.png 15
Gambar 3.2.12 - Memastikan foldernya sudah terbuat.png..... 16
Gambar 3.2.13 - Salin file data.txt dari Windows ke dalam container NameNode.png 17
Gambar 3.2.14 - Pindahkan (upload) file tersebut dari container ke dalam HDFS.png..... 17
Gambar 3.2.15 - Tampilkan daftar file di direktori HDFS untuk memastikan file sudah ter-upload.png..... 17
Gambar 3.2.16 - Tampilkan isi file data.txt yang ada di HDFS.png 18
Gambar 3.2.17 - Buat salinan file (copy).png..... 18
Gambar 3.2.18 - Ubah nama (rename atau move) file salinan tersebut menjadi data-backup.txt.png..... 19
Gambar 3.2.19 - Hapus (delete) file salinan yang baru saja di-rename tersebut.png 19
Gambar 3.2.20 - Mengecek status DataNode dan kapasitas HDFS_1.png.... 20
Gambar 3.2.21 - Mengecek status DataNode dan kapasitas HDFS_2.png.... 20
Gambar 3.2.22 - Membuat direktori Folder Praktikum.png..... 21

Gambar 3.2.23 - Memindahkan file data.txt dari folder lama ke folder yang baru.png 21
Gambar 3.2.24 - Menghapus folder Praktikum yang lama beserta isinya agar rapi.png 22
Gambar 3.2.25 - Verifikasi akhir (Menampilkan seluruh isi folder mahasiswa).png 22

BAB I
PENDAHULUAN

1.1 Tujuan Praktikum

Praktikum ini dilaksanakan dengan tujuan agar mahasiswa mampu:

1. Memahami konsep dasar dari arsitektur Apache Hadoop dan *Hadoop Distributed File System* (HDFS).
2. Membedakan fungsi serta peran krusial antara NameNode dan DataNode dalam klaster.
3. Melakukan instalasi dan konfigurasi lingkungan virtualisasi menggunakan WSL 2 dan Docker Desktop.
4. Menerapkan *deployment* klister Hadoop *single-node* berbasis *container* melalui Docker Compose.
5. Memantau kesehatan sistem terdistribusi melalui akses NameNode dan DataNode Web UI.
6. Mengeksekusi operasi dasar manajemen *file* pada ekosistem HDFS (mencakup pembuatan direktori, pengunggahan, pembacaan, penyalinan, pemindahan, hingga penghapusan *file*).
7. Membaca dan menganalisis laporan kondisi DataNode beserta utilitas kapasitas HDFS.

BAB II
DASAR TEORI

2.1 Konsep Apache Hadoop & HDFS

Apache Hadoop merupakan kerangka kerja (framework) berbasis open-source yang dirancang khusus untuk memproses dan menyimpan kumpulan data berskala masif (Big Data) secara terdistribusi melintasi klaster komputer. Ekosistem Hadoop terdiri atas beberapa komponen utama, di antaranya adalah HDFS untuk penyimpanan, YARN untuk penjadwalan sumber daya, dan MapReduce untuk pemrosesan data. HDFS (Hadoop Distributed File System) bertindak sebagai sistem file utama yang mampu memecah file berukuran raksasa menjadi blok-blok kecil, lalu menyebarkannya ke berbagai node pekerja (DataNode). Arsitektur ini memungkinkan HDFS beroperasi dengan keandalan (reliability) yang tinggi pada perangkat keras standar.

2.2 NameNode, DataNode & Replication Factor

Arsitektur HDFS menganut sistem *Master-Slave*. NameNode bertindak sebagai *Master* yang bertugas murni mengelola *metadata*, yaitu mencatat struktur direktori, hierarki *namespace*, serta melacak di *node* mana saja kepingan blok data disimpan. Di sisi lain, DataNode bertindak sebagai *Slave* atau *node* pekerja yang memikul beban utama untuk menyimpan wujud fisik dari blok-blok data tersebut secara aktual. Untuk memastikan data tidak hilang saat perangkat mengalami kerusakan, HDFS menggunakan mekanisme *Replication Factor* yang mengatur seberapa banyak salinan cadangan dari sebuah blok data disebar. Pada praktikum *single-node* ini, nilai *Replication Factor* ditetapkan pada angka 1 karena simulasi hanya menggunakan satu buah DataNode aktif.

BAB III
PEMBAHASAN

3.1 Langkah-Langkah Praktikum dan Command

Dalam pelaksanaan praktikum ini, terdapat beberapa tahapan instruksi yang dieksekusi melalui terminal PowerShell:

1. Pembuatan Direktori dan Konfigurasi Dasar

   Tahapan dimulai dengan pembuatan direktori kerja bigdata-hadoop di penyimpanan lokal. Di dalamnya, disusun file docker-compose.yml untuk orkestrasi *container*, serta file konfigurasi HDFS (core-site.xml dan hdfs-site.xml) yang ditempatkan pada folder conf.

   Berikut isi dari ketiga file:

   - core-site.xml

```
<?xml version="1.0"?>
<configuration>
<property>
<name>fs.defaultFS</name>
<value>hdfs://namenode:8020</value>
</property>
</configuration>
```

   - hdfs-site.xml

```
<?xml version="1.0"?>
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///tmp/hadoop-data/namenode</value>
  </property>
```

```
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///tmp/hadoop-data/datanode</value>
  </property>
  <property>
    <name>dfs.namenode.http-address</name>
    <value>0.0.0.0:9870</value>
  </property>
  <property>
    <name>dfs.datanode.http.address</name>
    <value>0.0.0.0:9864</value>
  </property>
</configuration>
```

- docker-compose.yml

```
services:
  namenode:
    image: apache/hadoop:3.4.3
    container_name: namenode
    hostname: namenode
    user: root
    command: >
      bash -c "
      mkdir -p /tmp/hadoop-data/namenode &&
      chown -R hadoop:hadoop /tmp/hadoop-data/namenode
&&
      if [ ! -d /tmp/hadoop-data/namenode/current ]; then
        su -s /bin/bash hadoop -c '/opt/hadoop/bin/hdfs
namenode -format -force -nonInteractive';
      fi &&
      su -s /bin/bash hadoop -c '/opt/hadoop/bin/hdfs
namenode'
      "
    ports:
      - "9870:9870"
```

```
      - "8020:8020"
    volumes:
      -          ./conf/core-site.xml:/opt/hadoop/etc/hadoop/core-
site.xml:ro
      -          ./conf/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-
site.xml:ro
      - namenode-data:/tmp/hadoop-data

  datanode:
    image: apache/hadoop:3.4.3
    container_name: datanode
    hostname: datanode
    user: root
    command: >
      bash -c "
      mkdir -p /tmp/hadoop-data/datanode &&
      chown -R hadoop:hadoop /tmp/hadoop-data/datanode
&&
      su -s /bin/bash hadoop -c '/opt/hadoop/bin/hdfs
datanode'
      "
    ports:
      - "9864:9864"
    volumes:
      -          ./conf/core-site.xml:/opt/hadoop/etc/hadoop/core-
site.xml:ro
      -          ./conf/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-
site.xml:ro
      - datanode-data:/tmp/hadoop-data
    depends_on:
      - namenode

volumes:
  namenode-data:
  datanode-data:
```

2. Validasi dan Penarikan Image

   Perintah docker compose config dijalankan guna memastikan tidak ada kesalahan sintaks pada kode YAML. Setelah validasi sukses, *image* instalasi ditarik dari *registry* menggunakan perintah docker pull apache/hadoop:3.4.3.

3. Mengaktifkan Klaster Hadoop

   Sistem dinyalakan di latar belakang dengan perintah docker compose up -d. Verifikasi status *container* (NameNode dan DataNode) dilakukan menggunakan perintah docker compose ps -a.

4. Pembuatan Direktori HDFS

   Pembuatan direktori tugas di dalam HDFS dieksekusi dengan mendelegasikan perintah lewat *user* Hadoop, yaitu docker exec -it –user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /mahasiswa/Muhammad_Anwar_Ismail/data dan /hasil.

5. Transfer Data dan Manipulasi File

   File data.txt ditransfer dari mesin *host* ke *container* via perintah docker cp, lalu diunggah permanen ke HDFS menggunakan utilitas -put. Pembacaan isi data diverifikasi menggunakan parameter -cat.

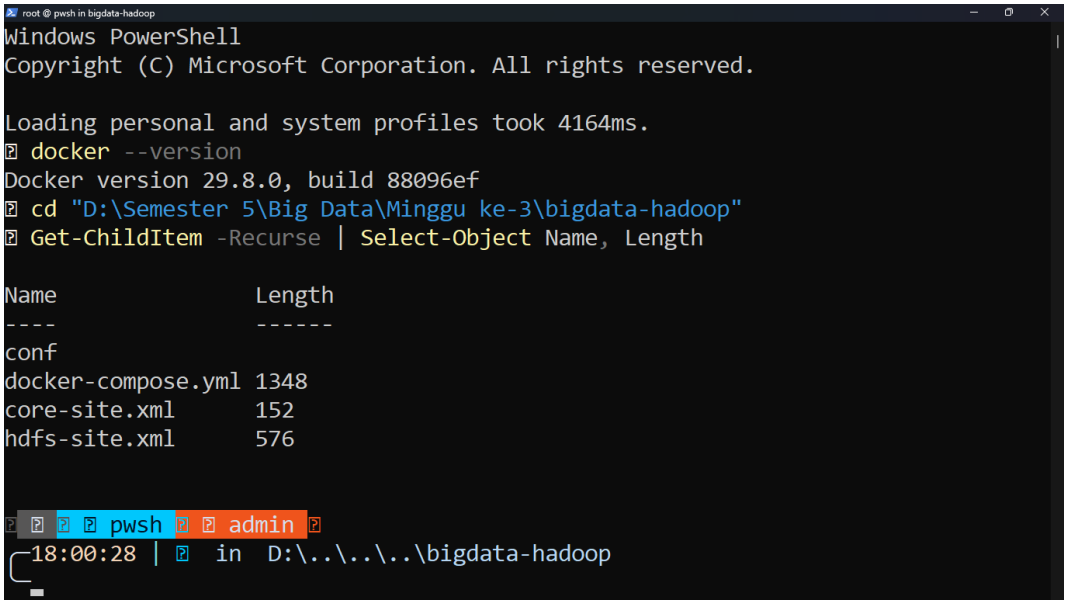
6. Manajemen *File* Terdistribusi

   Eksekusi penyalinan *file* dilakukan melalui parameter -cp, dilanjutkan dengan pengubahan nama *file* menggunakan parameter -mv, dan diakhiri dengan penghapusan *file backup* menggunakan parameter -rm.

7. Pengecekan Kesehatan Klaster

   Status kesehatan DataNode dan sisa kapasitas penyimpanan dicek secara menyeluruh menggunakan utilitas pelaporan hdfs dfsadmin -report.

3.2 Hasil Eksekusi dan Screenshot



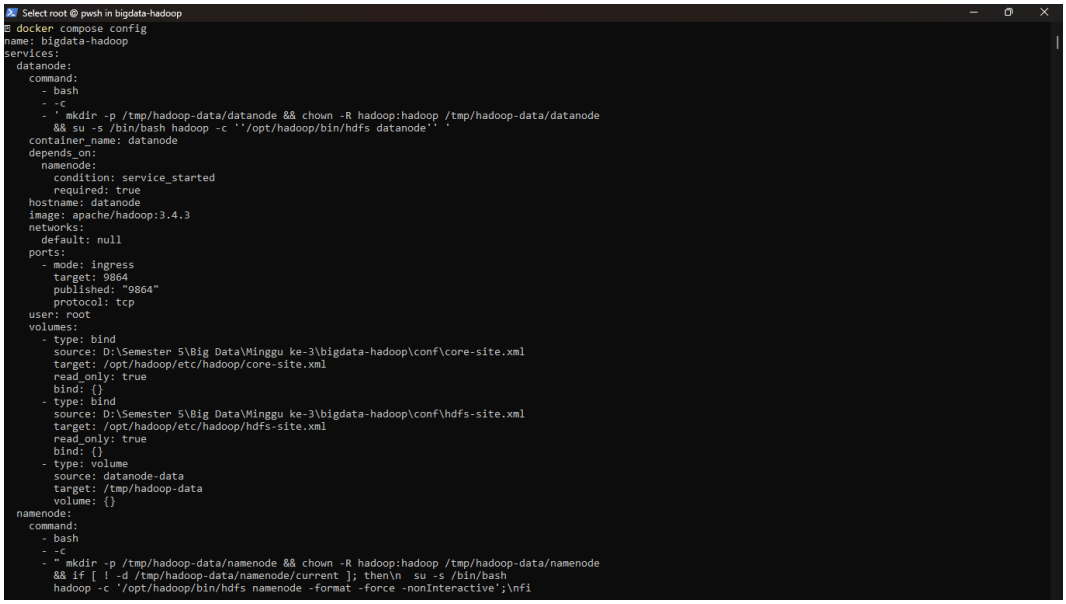
Gambar 3.2.1 - Menunjukkan struktur folder dan file yang berhasil dibuat.png

```
docker --version

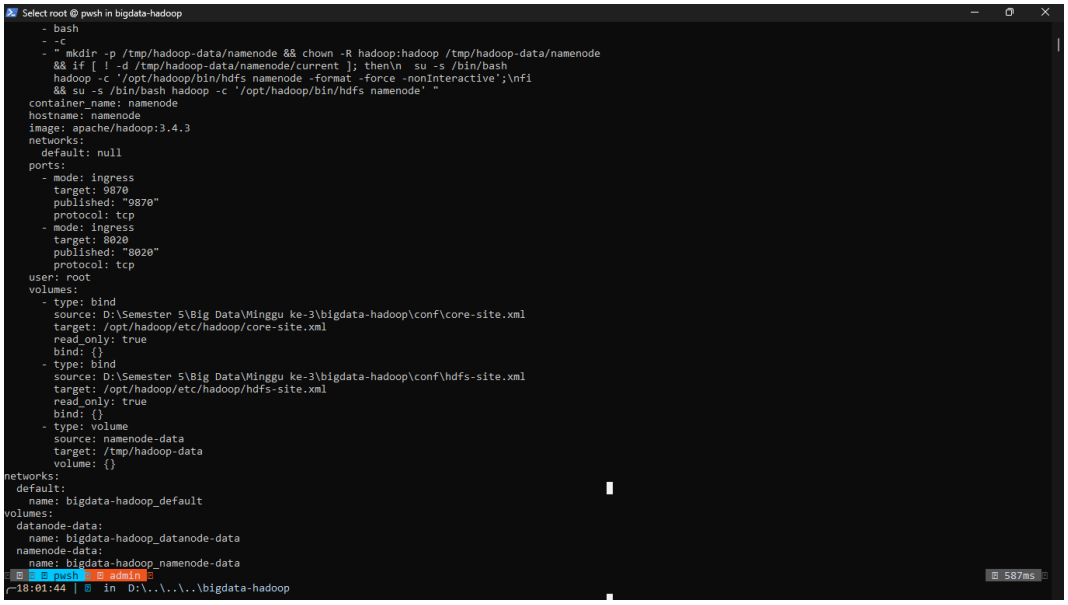
cd “D:\Semester 5\Big Data\Minggu ke-3\bigdata-hadoop”

Get-ChildItem -Recurse | Select-Object Name, Lenght
```

Berdasarkan hasil eksekusi pada terminal di atas, direktori kerja bigdata-hadoop beserta seluruh file pendukungnya telah berhasil dibentuk dengan sempurna. Pembuatan direktori ini dilakukan secara spesifik di dalam partisi tugas perkuliahan (D:\Semester 5\Big Data\Minggu ke-3). Keputusan pemilihan lokasi ini ditujukan semata-mata untuk mengoptimalkan manajemen proyek dan mencegah terjadinya penumpukan data pada partisi sistem operasi. Pembentukan struktur ini juga sepenuhnya didukung oleh Docker Compose yang bekerja mengandalkan sistem *relative path*, sehingga seluruh konfigurasi tetap berjalan secara optimal di mana pun direktori tersebut diletakkan. Pada direktori tersebut, file docker-compose.yml telah dipersiapkan bersama subdirektori conf yang menyimpan parameter krusial HDFS di dalam core-site.xml maupun hdfs-site.xml.



Gambar 3.2.2 - Validasi Docker Compose_1.png

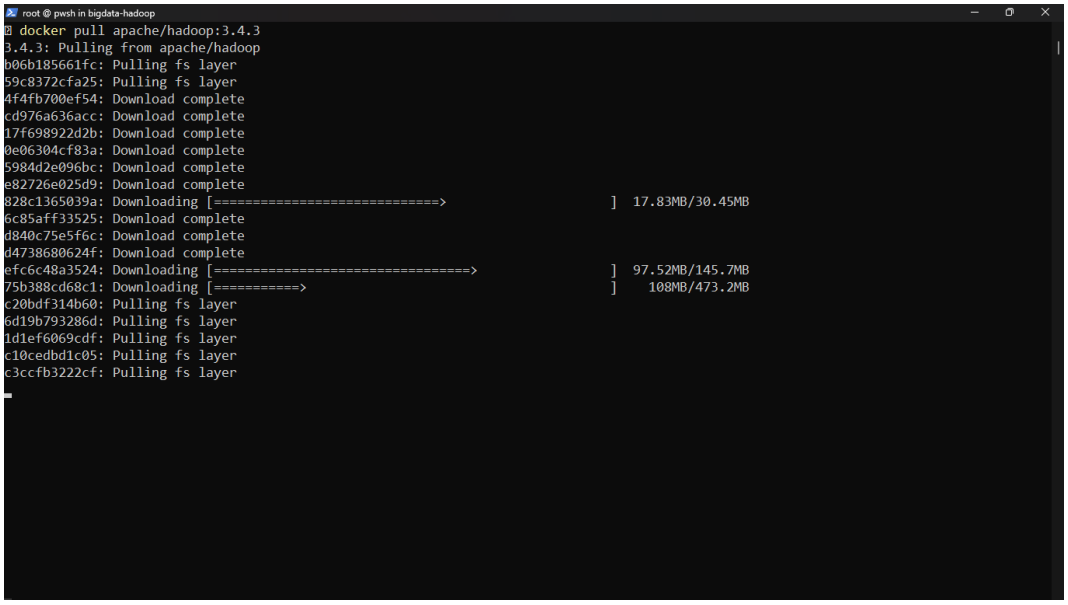


Gambar 3.2.3 - Validasi Docker Compose_2.png

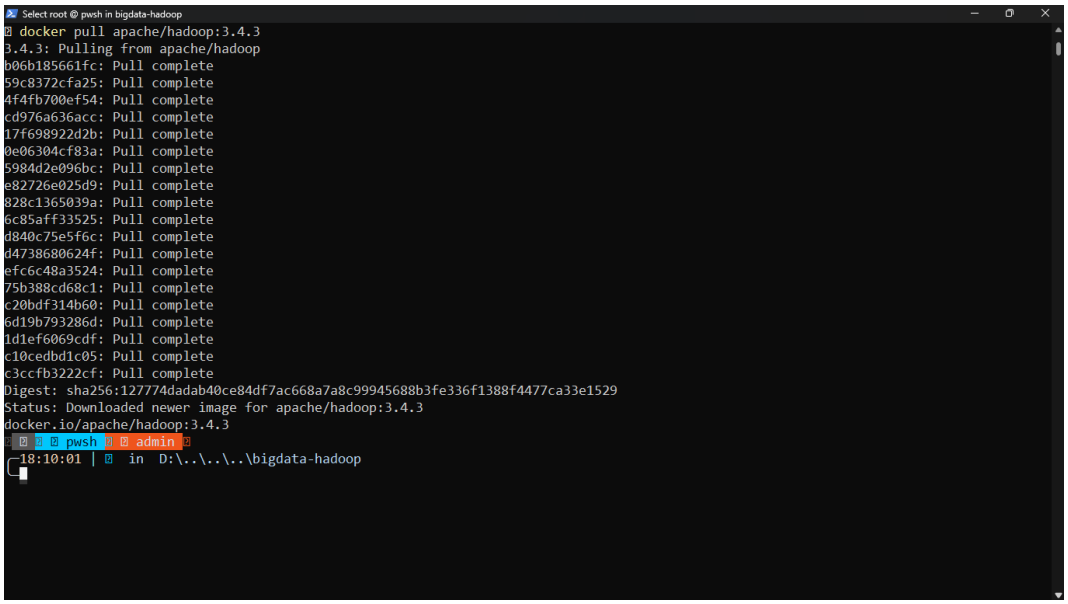
```
docker compose config
```

Proses validasi selanjutnya dijalankan menggunakan perintah docker compose config guna memverifikasi keabsahan struktur kode YAML yang telah dibuat. Tampilan layar terminal di atas memperlihatkan secara rinci hasil evaluasi *Docker Engine* yang tereksekusi dengan mulus tanpa adanya satupun

laporan *error* atau peringatan kegagalan. Keberhasilan validasi ini memberikan jaminan mutlak bahwa seluruh penulisan sintaks, definisi instruksi *bash* pada *container*, alokasi pemetaan *port*, serta pengaturan *volume data* telah tersusun secara presisi dan siap diimplementasikan untuk tahap *deployment* mesin virtual.



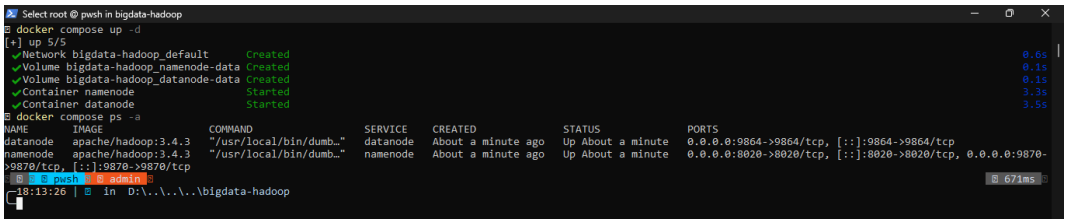
Gambar 3.2.4 - Proses Download Image_1.png



Gambar 3.2.5 - Proses Download Image_2.png

```
docker pull apache/hadoop:3.4.3
```

Langkah berikutnya berfokus pada penarikan (*pulling*) komponen instalasi Apache Hadoop dari repositori pusat Docker Hub menggunakan perintah docker pull apache/hadoop:3.4.3. Penetapan versi secara spesifik di angka 3.4.3 dilakukan dengan penuh kesengajaan demi memastikan keseragaman lingkungan pengembangan dengan instruksi modul praktikum. Pendekatan ini sekaligus berfungsi mengunci kestabilan sistem dari potensi kendala inkompatibilitas di kemudian hari. *Image* yang ditarik tersebut sudah membundel pustaka Java, sistem operasi dasar, serta fungsionalitas inti HDFS yang menjadi pondasi utama pembentukan klaster terdistribusi.



Gambar 3.2.6 - Menjalankan Cluster dan Verifikasi.png

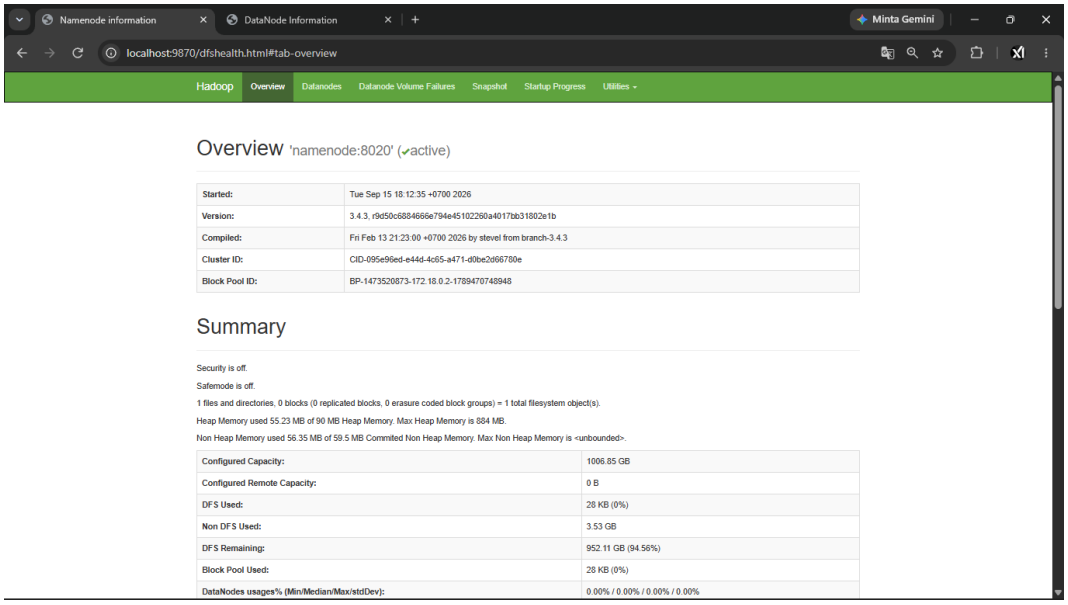
```
docker compose up -d

docker compose ps -a
```

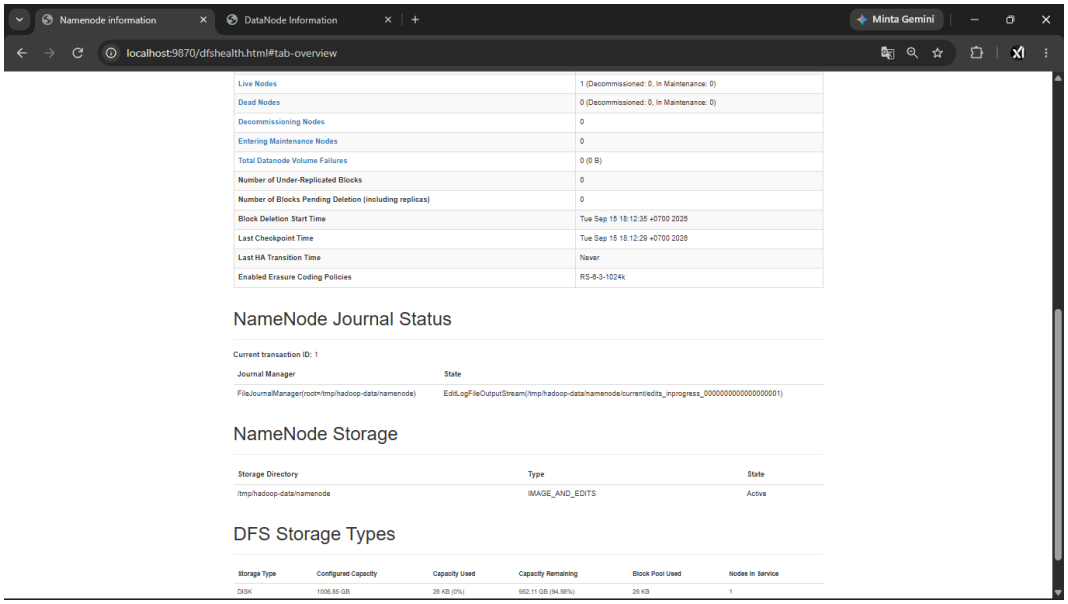
Tahapan instalasi mencapai puncaknya ketika klaster Hadoop berskala tunggal ini diaktifkan di latar belakang melalui eksekusi perintah docker compose up -d. Berdasarkan hasil tangkapan layar terminal, proses inisialisasi berjalan dengan sangat sempurna, yang ditandai dengan terbentuknya jaringan isolasi secara otomatis (bigdata-hadoop_default) serta pembuatan volume penyimpanan persisten untuk masing-masing *node* (namenode-data dan datanode-data).

Selanjutnya, guna memvalidasi keberhasilan operasional mesin, perintah docker compose ps -a turut dijalankan. Tabel *output* pada terminal mengonfirmasi secara meyakinkan bahwa kedua layanan krusial, yakni namenode dan datanode, telah sukses beroperasi dan mencapai status Up. Fakta ini membuktikan bahwa *daemon* HDFS telah berjalan secara utuh tanpa mengalami

kegagalan pemuatan (*crash*). Di samping itu, kolom *ports* juga memperlihatkan bahwa pemetaan jaringan ke sistem *host* lokal telah terhubung dengan presisi, seperti terbukanya *port* 9870 yang didedikasikan untuk akses antarmuka web NameNode, serta *port* 9864 untuk kontrol DataNode. Kondisi prima ini secara definitif memastikan bahwa klaster HDFS telah sepenuhnya siap untuk menerima dan mengeksekusi instruksi penyimpanan data terdistribusi.

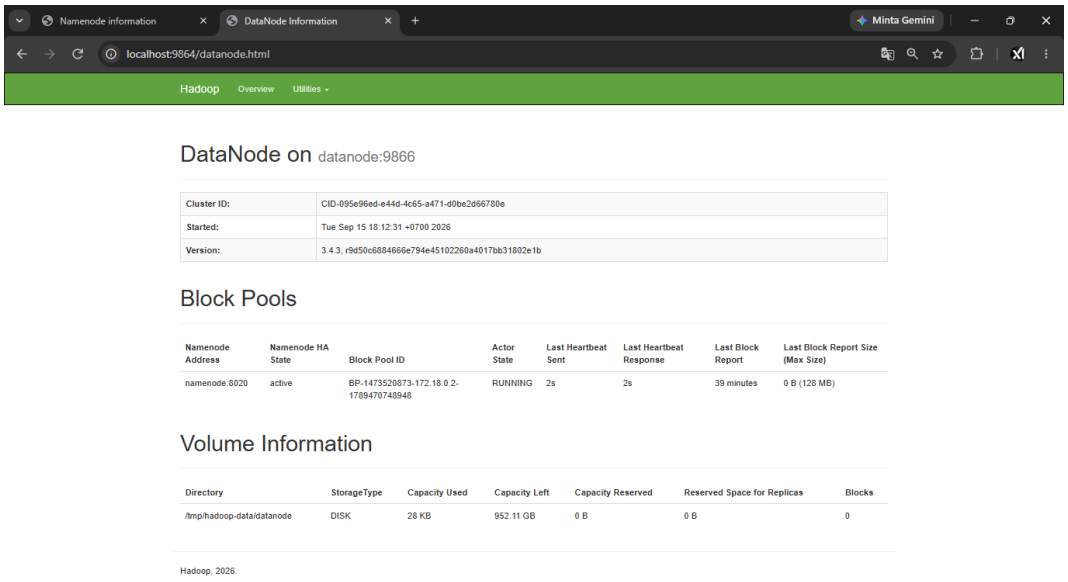


Gambar 3.2.7 - NameNode Web UI_1.png



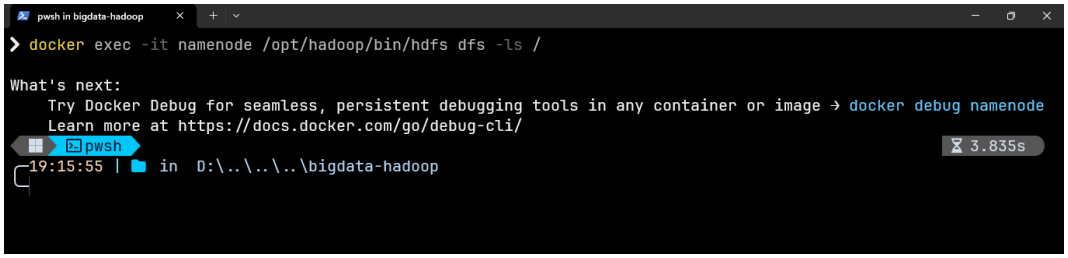
Gambar 3.2.8 - NameNode Web UI_2.png

Proses verifikasi selanjutnya beralih pada pengecekan antarmuka grafis atau *Web User Interface* (Web UI) bawaan Hadoop melalui peramban web (*browser*). Akses menuju Alamat localhost:9870 sukses menampilkan *dashboard* utama NameNode. Berdasarkan tangkapan layar tersebut, sistem menyajikan rekapitulasi metrik yang sangat komprehensif, mulai dari total kapasitas penyimpanan klaster (*Configured Capacity*), tingkat pemakaian ruang (*DFS Used*), hingga konfirmasi keberadaan satu *Live Node*. Kehadiran *Live Node* ini menandakan bahwa DataNode secara aktif dan konsisten mengirimkan sinyal *heartbeat* ke NameNode.



Gambar 3.2.9 - DataNode Web UI.png

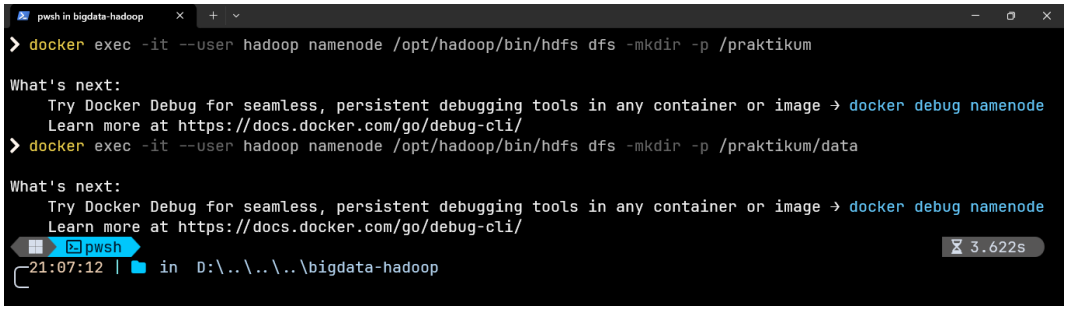
Di sisi lain, pengujian akses menuju alamat localhost:9864 berhasil membuka halaman antarmuka khusus DataNode. Halaman ini memvisualisasikan rincian teknis di level *node* pekerja, mencakup kondisi volume penyimpanan fisik dan log statistik blok data. Keberhasilan dalam memuat dan merender kedua antarmuka grafis ini membuktikan secara empiris bahwa proses manajemen *metadata* oleh NameNode dan pelaporan kapasitas penyimpanan oleh DataNode telah terintegrasi dan beroperasi secara harmonis di dalam *environment* Docker.



Gambar 3.2.10 - Melihat isi root HDFS.png

```
docker exec -it namenode /opt/Hadoop/bin/hdfs dfs -ls
```

Langkah praktik pengoperasian HDFS diawali dengan melakukan inspeksi terhadap direktori akar (*root*) menggunakan perintah hdfs dfs -ls /. Tangkapan layar di atas memperlihatkan eksekusi perintah yang langsung mengembalikan *prompt* kosong tanpa pesan *error*. Secara teknis, ketiadaan *output* ini memvalidasi bahwa mesin HDFS baru saja diformat secara bersih dan belum menampung struktur data maupun *file* apa pun, persis menyerupai perilaku utilitas direktori pada sistem operasi keluarga Unix yang tidak menampilkan data pada folder yang benar-benar kosong.



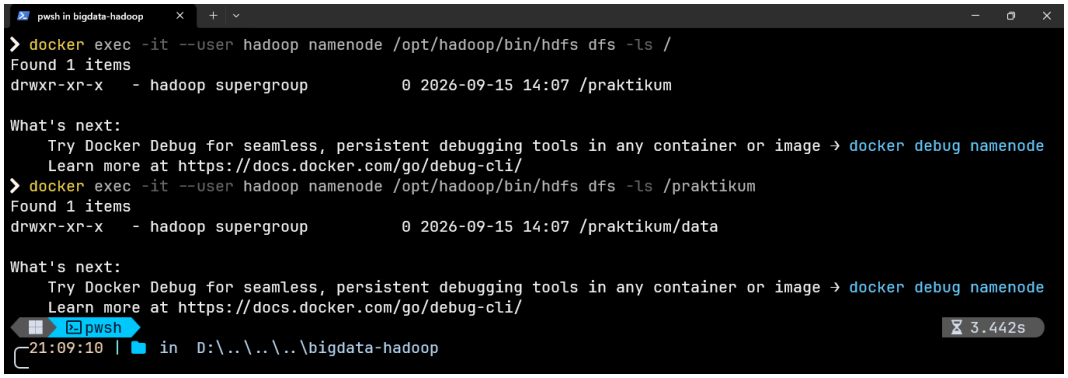
Gambar 3.2.11 - Membuat folder praktikum dan data.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /praktikum

docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /praktikum/data
```

Tahapan berikutnya berfokus pada modifikasi *filesystem* HDFS melalui instruksi pembentukan struktur direktori bertingkat menggunakan perintah hdfs dfs

-mkdir -p /praktikum/data. Parameter -p (*parents*) memegang peran operasional yang krusial, sebab ia menginstruksikan sistem untuk membangun direktori induk (/praktikum) secara otomatis sebelum membuat direktori anak (/data). Poin analisis paling penting pada eksekusi ini adalah penggunaan *flag* tambahan --user hadoop. Modifikasi ini wajib dilakukan untuk merespons fitur keamanan bawaan HDFS, mengingat *user root* bawaan dari *container* Linux tidak secara otomatis memiliki hak modifikasi (*write permission*) terhadap direktori *root* HDFS. Dengan mendelegasikan perintah lewat identitas hadoop, otorisasi berhasil dipenuhi.

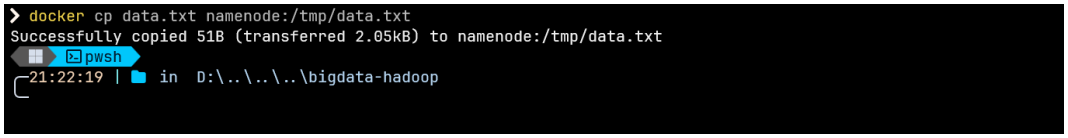


Gambar 3.2.12 - Memastikan foldernya sudah terbuat.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /

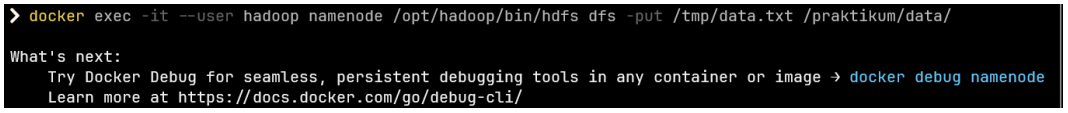
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum
```

Guna memastikan integritas arsitektur direktori yang telah dibuat, proses verifikasi dieksekusi ulang menggunakan perintah hdfs dfs -ls. Berbeda dengan kondisi awal, hasil pemanggilan data pada letak direktori / dan /praktikum kini secara eksplisit menjabarkan susunan entitas folder yang baru diregistrasi beserta seluruh metadatanya. Pada *output* terminal, atribut drwxr-xr-x memberikan Gambaran jelas bahwa system mengenalinya sebagai sebuah direktori (d) yang memberlakukan hak baca, tulis, dan eksekusi penuh bagi pemiliknya (*user hadoop*), namun dengan pengamanan ketat berupa hak baca saja untuk pihak luar. Rangkaian keberhasilan ini mengonfirmasi bahwa NameNode telah sukses mencatat dan memutakhirkan perubahan *namespace* tersebut ke dalam repositori *metadata*-nya.



Gambar 3.2.13 - Salin file data.txt dari Windows ke dalam container NameNode.png

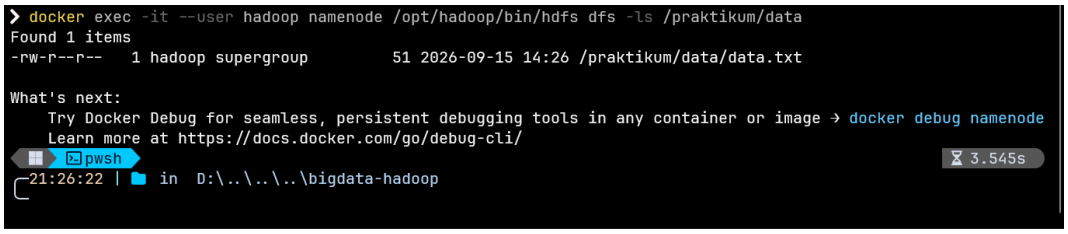
```
docker cp data.txt namenode:/tmp/data.txt
```



Gambar 3.2.14 - Pindahkan (upload) file tersebut dari container ke dalam HDFS.png

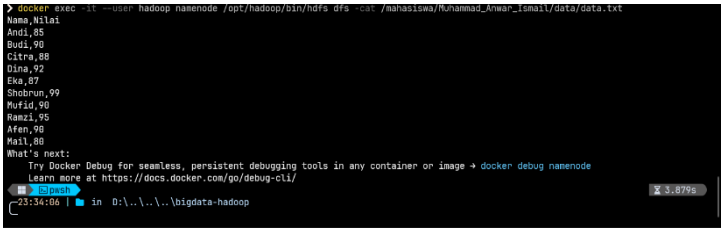
```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -put /tmp/data.txt /praktikum/data/
```

Untuk memasukkan data ke dalam HDFS, file data.txt yang ada di laptop (Windows) harus dipindahkan terlebih dahulu ke dalam *container* Linux. Hal ini dilakukan menggunakan perintah docker cp, yang menyalin file tersebut ke dalam folder sementara (/tmp) di *container* NameNode. Setelah file masuk ke dalam *container*, langkah selanjutnya adalah mengunggahnya ke HDFS menggunakan perintah hdfs dfs -put. File tersebut diletakkan pada direktori /praktikum/data/ yang sudah kita buat sebelumnya. Proses dua tahap ini harus dilakukan karena HDFS berjalan secara terisolasi di dalam Docker dan tidak bisa mengambil file langsung dari penyimpanan lokal Windows.



Gambar 3.2.15 - Tampilkan daftar file di direktori HDFS untuk memastikan file sudah ter-upload.png

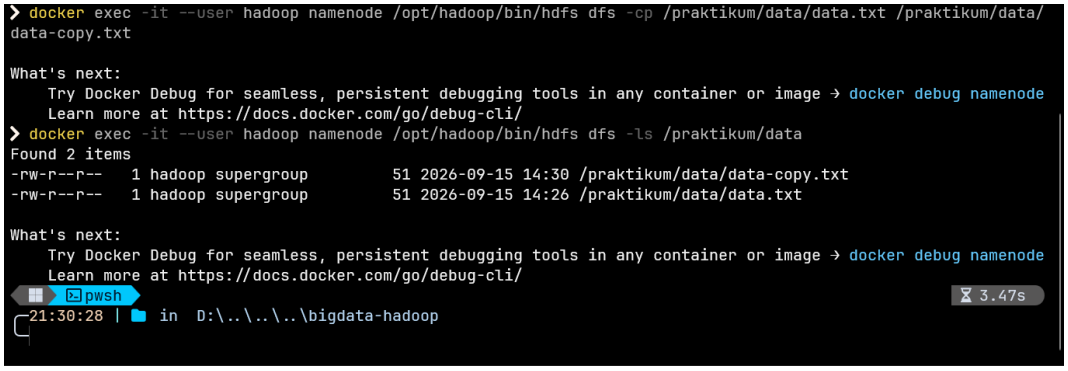
```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum/data
```



Gambar 3.2.16 - Tampilkan isi file data.txt yang ada di HDFS.png

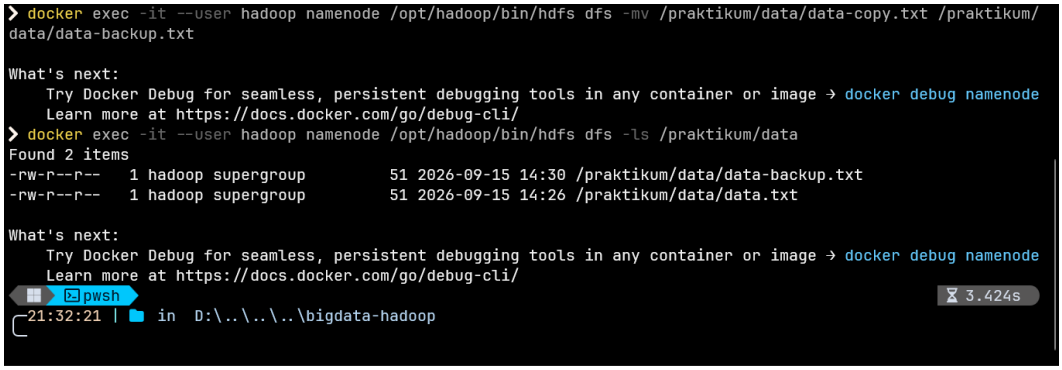
```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cat /mahasiswa/Muhammad_Anwar_Ismail/data/data.txt
```

Setelah proses unggah selesai, perintah hdfs dfs -ls dijalankan kembali untuk mengecek isi direktori /praktikum/data/. *Output* di terminal memperlihatkan bahwa file data.txt sudah berhasil masuk ke HDFS, lengkap dengan informasi ukuran file dan waktu pembuatannya. Selanjutnya, untuk memastikan isi data tidak ada yang terpotong saat proses unggah, perintah hdfs dfs -cat digunakan. Perintah ini berfungsi untuk membaca dan menampilkan isi file teks secara langsung di layar terminal. Hasilnya, 10 baris data CSV berisi nama dan nilai mahasiswa berhasil ditampilkan dengan sempurna sesuai dengan file aslinya.



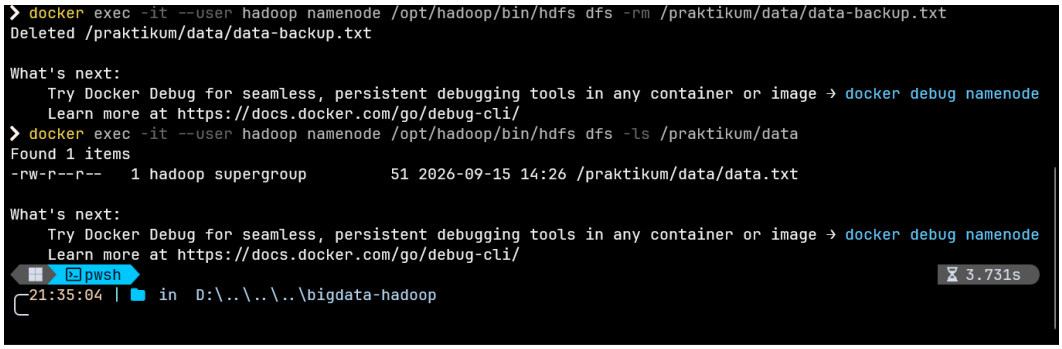
Gambar 3.2.17 - Buat salinan file (copy).png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cp /praktikum/data/data.txt /praktikum/data/data-copy.txt
```



Gambar 3.2.18 - Ubah nama (rename atau move) file salinan tersebut menjadi data-backup.txt.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mv /praktikum/data/data-copy.txt /praktikum/data/data-backup.txt
```

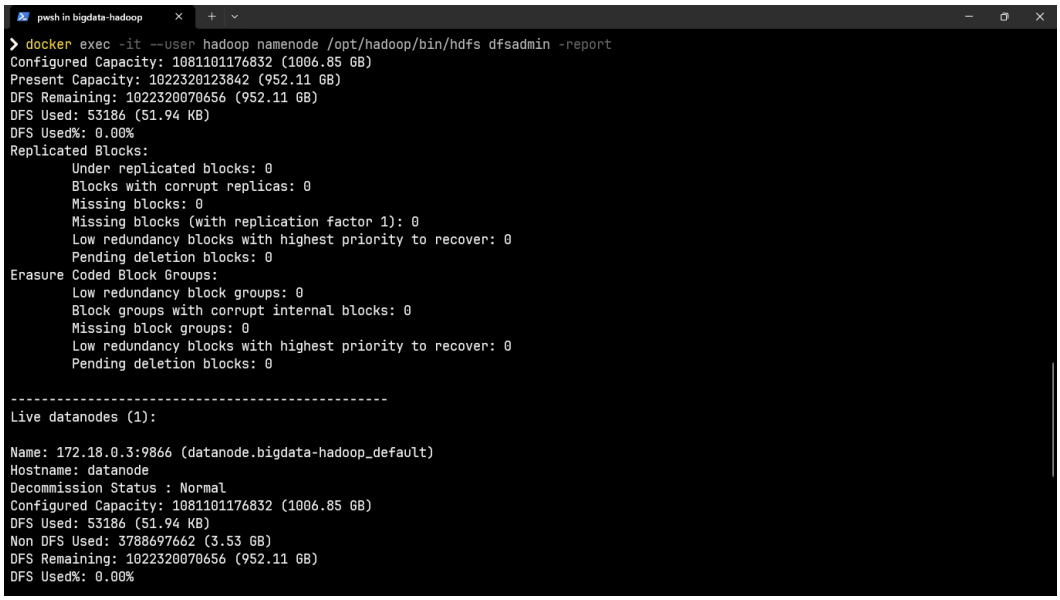


Gambar 3.2.19 - Hapus (delete) file salinan yang baru saja di-rename tersebut.png

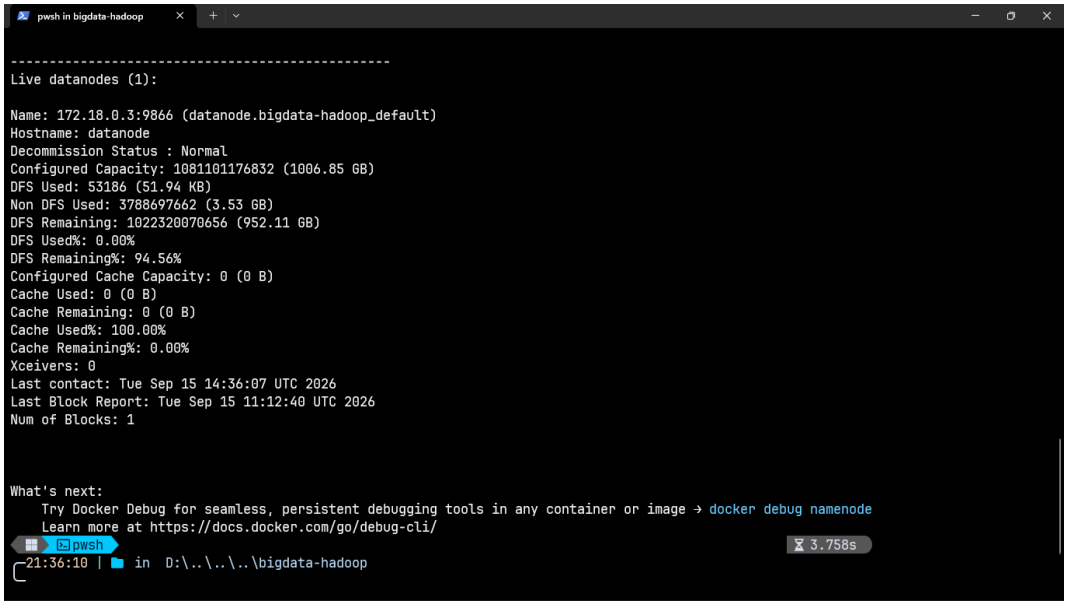
```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -rm /praktikum/data/data-backup.txt
```

Praktikum dilanjutkan dengan mencoba perintah dasar manajemen file yang ada di HDFS. Pertama, file disalin menggunakan perintah hdfs dfs -cp untuk menghasilkan file duplikat bernama data-copy.txt. Kedua, nama file hasil salinan tersebut diubah menjadi data-backup.txt menggunakan perintah hdfs dfs -mv (*rename*). Terakhir, file *backup* tersebut dihapus secara permanen menggunakan perintah hdfs dfs -rm. Keterangan status *Deleted* pada terminal menandakan bahwa eksekusi berhasil. Rangkaian uji coba ini membuktikan bahwa HDFS mendukung

operasi dasar (*copy*, *rename*, dan *delete*) layaknya sistem file biasa, dan NameNode sanggup mencatat semua perubahan struktur file tersebut secara instan.



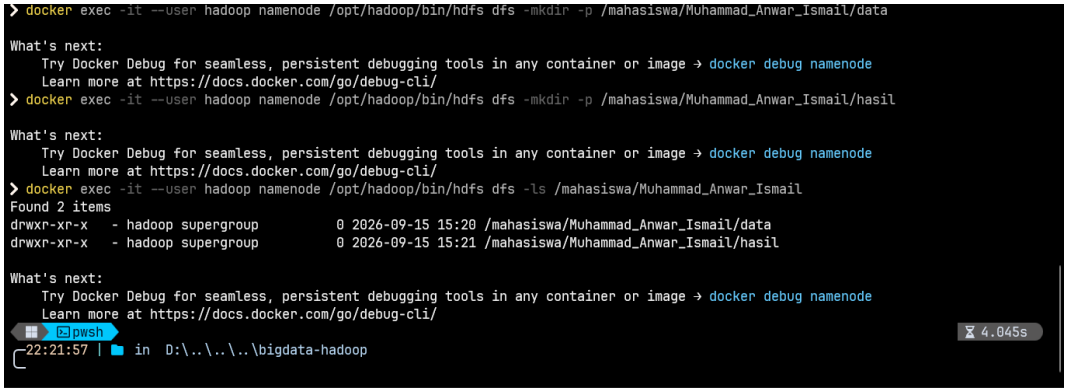
Gambar 3.2.20 - Mengecek status DataNode dan kapasitas HDFS_1.png



Gambar 3.2.21 - Mengecek status DataNode dan kapasitas HDFS_2.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfsadmin -report
```

Sebagai langkah terakhir dari praktikum ini, dilakukan pengecekan status keseluruhan klaster HDFS menggunakan perintah hdfs dfsadmin -report. Output pada terminal langsung menampilkan ringkasan informasi sistem secara detail, mulai dari total kapasitas penyimpanan (*Configured Capacity*), ruang yang sudah terpakai (*DFS Used*), hingga sisa ruang yang masih tersedia. Informasi utama yang menjadi fokus pada pengecekan ini adalah metrik *Live datanodes*. Angka 1 pada baris tersebut mengonfirmasi bahwa DataNode sudah menyala, terhubung dengan baik ke NameNode, dan siap untuk menyimpan data. Hasil laporan ini secara langsung membuktikan bahwa konfigurasi klaster Hadoop *single-node* menggunakan Docker Desktop telah berhasil diimplementasikan dengan benar sesuai tujuan praktikum.



Gambar 3.2.22 - Membuat direktori Folder Praktikum.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /mahasiswa/Muhammad_Anwar_Ismail/data

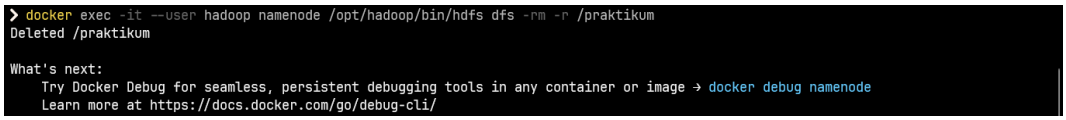
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /mahasiswa/Muhammad_Anwar_Ismail/hasil

docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /mahasiswa/Muhammad_Anwar_Ismail
```



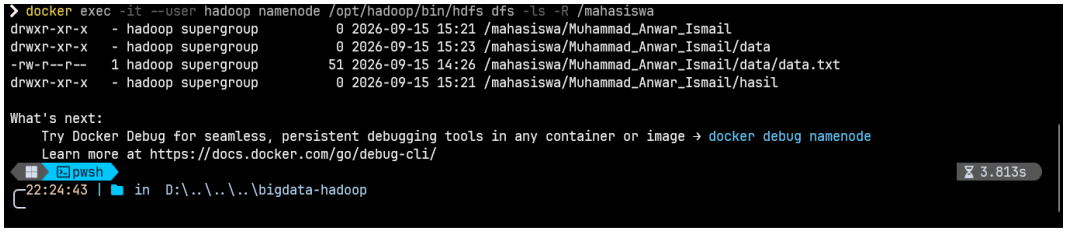
Gambar 3.2.23 - Memindahkan file data.txt dari folder lama ke folder yang baru.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mv /praktikum/data/data.txt /mahasiswa/Muhammad_Anwar_Ismail/data/
```



Gambar 3.2.24 - Menghapus folder Praktikum yang lama beserta isinya agar rapi.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -rm -r /praktikum
```



Gambar 3.2.25 - Verifikasi akhir (Menampilkan seluruh isi folder mahasiswa).png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls -R /mahasiswa
```

Sebagai langkah penyempurnaan akhir, dilakukan penyesuaian struktur direktori agar selaras dengan instruksi spesifik pada poin Tugas Praktikum. Struktur direktori /praktikum yang sebelumnya digunakan selama masa eksplorasi awal (tutorial) dimigrasikan ke dalam format direktori resmi yang diminta, yaitu /mahasiswa/Muhammad_Anwar_Ismail/.

Proses migrasi ini mendemonstrasikan kapabilitas manajemen file HDFS secara utuh. Pertama, direktori baru (/data dan /hasil) dibentuk menggunakan perintah hdfs dfs -mkdir -p. Selanjutnya, file data utama dipindahkan secara aman dari folder lama ke folder target menggunakan perintah hdfs dfs -mv. Setelah dipastikan data berpindah dengan selamat, lingkungan penyimpanan dibersihkan dari *file* sementara (*temporary workspace*) dengan menghapus seluruh folder /praktikum secara rekursif menggunakan perintah hdfs dfs -rm -r. Keseluruhan proses ini divalidasi pada tahap akhir menggunakan perintah hdfs dfs -ls -R

/mahasiswa, yang langsung menampilkan pemetaan *tree* direktori secara lengkap. Hasil verifikasi tersebut membuktikan bahwa file data.txt telah berada di posisi yang tepat tanpa kehilangan data sedikit pun.

- CHECKLIST KEBERHASILAN PRAKTIKUM

[√] WSL 2 aktif

[√] Docker Desktop berjalan

[√] docker run hello-world berhasil

[√] Image apache/hadoop:3.4.3 berhasil di-download

[√] NameNode berstatus Up

[√] DataNode berstatus Up

[√] localhost:9870 dapat dibuka

[√] Direktori /praktikum berhasil dibuat

[√] File berhasil di-upload ke HDFS

[√] File dapat dibaca dengan -cat

[√] File berhasil di-copy/move/delete

[√] dfsadmin -report menampilkan DataNode aktif

3.3 Jawaban Pertanyaan Analisis

10. Apa fungsi NameNode dan DataNode?

    NameNode berfungsi secara eksklusif sebagai otak pengelola tata letak file dan struktur direktori HDFS. Sementara itu, DataNode bertugas murni sebagai tempat node yang menyediakan ruang penyimpanan fisik untuk potongan-potongan data tersebut..

11. Mengapa NameNode menyimpan metadata, sedangkan DataNode menyimpan block data?

    Pembagian peran ini dilakukan semata-mata demi efisiensi performa. NameNode bertindak layaknya daftar isi buku yang butuh kecepatan akses super tinggi lewat RAM. Jika NameNode ikut memikul beban menampung file aslinya, memori pasti akan langsung jebol dan seluruh sistem bakal lumpuh.

Makanya, tugas berat I/O data mentahnya diserahkan penuh kepada barisan DataNode.

12. Apa yang dimaksud dengan block pada HDFS?

    Block merupakan kepingan atau satuan terkecil penyimpanan di dalam ekosistem HDFS. Setiap kali ada file raksasa yang masuk, sistem otomatis mencacahnya menjadi beberapa bagian kecil (biasanya berukuran 128 MB per keping), lalu menyebarkannya ke berbagai titik DataNode.

13. Apa fungsi replication factor?

    Fitur ini bertugas mengatur jumlah salinan cadangan dari setiap kepingan data. Tujuannya sangat jelas yakni untuk menjamin keamanan data. Seandainya ada satu server yang mendadak *down* atau rusak, kita tetap punya jaminan akses ke data utuh lewat salinannya di mesin lain.

14. Mengapa *replication factor* praktikum ditetapkan 1?

    Kita menyetel nilainya di angka 1 karena simulasi cluster Docker kita dirancang sangat minimalis dengan satu DataNode aktif saja. Keputusan ini sangat membantu menghemat pemakaian RAM laptop kita. Apabila kita paksa di angka yang lebih tinggi, sistem Hadoop pasti akan terus melontarkan *error* karena kebingungan mencari mesin cadangan.

15. Apa yang terjadi jika DataNode mati ketika *replication factor* = 1?

    Akses menuju data tersebut akan terputus total. Mengingat kita sama sekali tidak punya salinan cadangan, kerusakan permanen pada node tunggal itu otomatis membuat seluruh data kita lenyap tak bersisa.

16. Apa perbedaan path C:... dengan /... pada HDFS?

    Path C:\... murni mewakili lokasi fisik pada sistem operasi lokal Windows kita. Di sisi lain, path /... merupakan struktur direktori virtual yang terisolasi dan sepenuhnya dikendalikan oleh NameNode HDFS. Kita wajib menggunakan perintah terminal khusus untuk bisa menembus dan melihat isi area HDFS tersebut.

17. Mengapa HDFS sesuai untuk penyimpanan Big Data?

    Sistem ini terbukti sangat tangguh menangani ukuran file luar biasa besar berkat kemampuannya memecah beban kerja. HDFS dirancang berjalan mulus di

spesifikasi komputer standar, sangat gampang diperluas kapasitasnya, dan punya jaminan keamanan data yang kuat berkat fitur replikasi otomatisnya.

18. Bagaimana perubahan jumlah DataNode memengaruhi HDFS?

    Semakin banyak DataNode yang kita gabungkan, ruang tampung total otomatis melesat naik. Kinerja proses baca dan tulis juga jauh lebih cepat karena sistem mengerjakannya secara bersamaan, sekaligus membuka peluang bagi kita untuk memperbanyak jumlah salinan pengaman data.

BAB IV
KESIMPULAN

4.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi Apache Hadoop dan HDFS dapat berjalan optimal di lingkungan sistem operasi Windows menggunakan bantuan WSL 2 dan Docker Desktop. Klaster berskala Tunggal (ingle-node) yang terdiri atas satu NameNode dan satu DataNode berhasil dibangun dan dikonfigurasi dengan baik menggunakan Docker Compose.

Seluruh antarmuka *Web-UI* sukses diakses untuk memantau status Kesehatan *node* secara *real-time*. Selain itu, eksekusi perintah dasar HDFS mulai dari membuat direktori (mkdir), mengunggah data (mengupload), membaca isi teks (cat), menyalin (cp), mengubah nama (mv), hingga menghapus file (rm) yang telah berhasil dilakukan sepenuhnya tanpa error. Pemahaman mendasar mengenai pengoperasian baris perintah (*command line)* dan manajemen file terdistribusi ini menjadi pondasi yang sangat penting sebelum melangkah ke materi pemrosesan Big Data lanjutan seperti YARN, MapReduce, maupun Apache Spark.

LAMPIRAN A
COMMAND UTAMA

JENIS	COMMAND UTAMA
Validasi Docker	docker compose config
Download Image	docker pull apache/hadoop:3.4.3
Menyalakan Klaster	docker compose up -d
Melihat Status Container	docker compose ps -a
Mematikan Klaster	docker compose down
List Direktori HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /
Membuat Direktori HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /mahasiswa/Muhammad_Anwar_Ismail/data
Upload File HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -put /tmp/data.txt /mahasiswa/Muhammad_Anwar_Ismail/data/
Membaca File HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cat /mahasiswa/Muhammad_Anwar_Ismail/data/data.txt
Copy File HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cp /mahasiswa/Muhammad_Anwar_Ismail/data /data.txt

	/mahasiswa/Muhammad_Anwar_Ismail/data/data-copy.txt
Rename File HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mv /mahasiswa/Muhammad_Anwar_Ismail/data/data-copy.txt /mahasiswa/Muhammad_Anwar_Ismail/data/data-backup.txt
Hapus File HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -rm /mahasiswa/Muhammad_Anwar_Ismail/data/data-backup.txt
Laporan Status HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfsadmin -report

LAMPIRAN B

PORT YANG DIGUNAKAN

Port	Komponen	Digunakan
9870	NameNode	Untuk mengakses halaman *Web UI* (antarmuka grafis) milik NameNode via *browser*.
8020	NameNode	Sebagai *port* komunikasi layanan internal HDFS (RPC) milik NameNode.
9864	DataNode	Untuk mengakses halaman *Web UI* milik DataNode.